

Lab 13 Answers

Yanqiao Chen

Department of Computer Science and Engineering
Southern University of Science and Technology

12412115@mail.sustech.edu.cn

May 30, 2026

1 Question 1

1.1 a)

The computation of M on $w = 111$ is

$$q_0 111 \perp \vdash 1 q_1 11 \perp \vdash 11 q_0 1 \perp \vdash 111 q_1 \perp \vdash 111 \perp q_{\text{acc}}.$$

Thus the 5×5 tableau is as follows:

q_0	1	1	1	$\perp$
1	q_1	1	1	$\perp$
1	1	q_0	1	$\perp$
1	1	1	q_1	$\perp$
1	1	1	$\perp$	q_{acc}

Table 1: The 5×5 tableau for M on input 111.

1.2 b)

Let $x_{i,j,s}$ be the Boolean variable which is true iff the cell in row i and column j of the tableau contains symbol/state s .

For the input $w = 111$, the starting configuration is $q_0 111 \perp$, so

$$\phi_{\text{start}} = x_{1,1,q_0} \wedge x_{1,2,1} \wedge x_{1,3,1} \wedge x_{1,4,1} \wedge x_{1,5,\perp}.$$

The accepting condition is that q_{acc} appears somewhere in the tableau:

$$\phi_{\text{accept}} = \bigvee_{i=1}^5 \bigvee_{j=1}^5 x_{i,j,q_{\text{acc}}}.$$

In this tableau, this is satisfied because $x_{5,5,q_{\text{acc}}}$ is true.

2 Question 2

- a. illegal
- b. illegal
- c. legal
- d. legal
- e. legal
- f. legal
- g. legal

3 Question 3

We give a counterexample to show that checking only 2×2 windows is not enough. Consider a Turing machine with the following two transitions:

$$\delta(q, 0) = (r, 0, L), \quad \delta(q, 1) = (s, 1, L),$$

where $r \neq s$.

Now consider the following 2×3 window:

$$\begin{array}{ccc} 0 & q & 1 \\ r & 0 & 1 \end{array}$$

This window is illegal. In the first row, the machine is in state q and is scanning the symbol 1. Therefore the transition that should be used is $\delta(q, 1) = (s, 1, L)$, so the second row should contain state s in the left cell, not state r .

However, the two 2×2 subwindows both look legal. The left subwindow is

$$\begin{array}{cc} 0 & q \\ r & 0 \end{array}$$

which can appear as part of the legal 2×3 window

$$\begin{array}{ccc} 0 & q & 0 \\ r & 0 & 0 \end{array}$$

using the transition $\delta(q, 0) = (r, 0, L)$. The right subwindow is

$$\begin{array}{cc} q & 1 \\ 0 & 1 \end{array}$$

which can appear as part of the legal 2×3 window

$$\begin{array}{ccc} 0 & q & 1 \\ s & 0 & 1 \end{array}$$

using the transition $\delta(q, 1) = (s, 1, L)$.

Thus every 2×2 subwindow may be legal while the whole 2×3 window is illegal. Hence the Cook–Levin proof would fail with 2×2 windows, because such windows cannot always verify one complete transition of the Turing machine.

4 Question 4

First, $UHAMPATH \in NP$, because given a path from s to t , we can check in polynomial time that every vertex appears exactly once and that every consecutive pair of vertices is connected by an edge.

Now we reduce $HAMPATH$ to $UHAMPATH$. Let $\langle G, s, t \rangle$ be an instance of directed $HAMPATH$, where $G = (V, E)$. For each vertex $u \in V$, create three vertices u^{in}, u^{mid}, u^{out} in the undirected graph G' . Inside each gadget, add the two undirected edges

$$\{u^{in}, u^{mid}\}, \quad \{u^{mid}, u^{out}\}.$$

The gadget is shown below:



For each directed edge $(u, v) \in E$, add the undirected edge

$$\{u^{out}, v^{in}\}$$

to G' . Finally, set the start and end vertices in G' to be

$$s' = s^{in}, \quad t' = t^{out}.$$

This construction is polynomial, since each vertex is replaced by three vertices and each directed edge gives one undirected edge.

If G has a directed Hamiltonian path

$$s = v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_n = t,$$

then G' has the undirected Hamiltonian path

$$v_1^{in}, v_1^{mid}, v_1^{out}, v_2^{in}, v_2^{mid}, v_2^{out}, \dots, v_n^{in}, v_n^{mid}, v_n^{out}.$$

The edges inside each gadget exist by construction, and the edge $\{v_i^{out}, v_{i+1}^{in}\}$ exists because $(v_i, v_{i+1}) \in E$.

Conversely, suppose G' has an undirected Hamiltonian path from s^{in} to t^{out} . For every vertex u , the vertex u^{mid} is adjacent only to u^{in} and u^{out} . Hence a Hamiltonian path must pass through the three vertices of each gadget consecutively, either as

$$u^{in}, u^{mid}, u^{out}$$

or in the reverse order. Since the whole path starts at s^{in} , the first gadget must be traversed from s^{in} to s^{out} . After that, the only way to move to another gadget is through an edge of the form $\{u^{out}, v^{in}\}$, so the next gadget is also entered at its v^{in} vertex and must be traversed as v^{in}, v^{mid}, v^{out} . Repeating this argument, all gadgets are traversed from their *in* vertex to their *out* vertex.

Therefore the order in which the gadgets appear gives a directed path in G : whenever the path in G' goes from u^{out} to v^{in} , the construction guarantees that $(u, v) \in E$. Since the Hamiltonian path in G' visits every gadget exactly once and goes from s^{in} to t^{out} , the corresponding path in G visits every vertex exactly once and goes from s to t . Thus G has a directed Hamiltonian path iff G' has an undirected Hamiltonian path from s' to t' .

Hence $UHAMPATH$ is NP-hard. Since $UHAMPATH \in NP$, it is NP-Complete.

5 Question 5

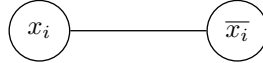
First, VERTEX COVER is in NP. Given a set C of at most k vertices, we can check in polynomial time whether every edge of G has at least one endpoint in C .

Now we reduce 3SAT to VERTEX COVER. Let

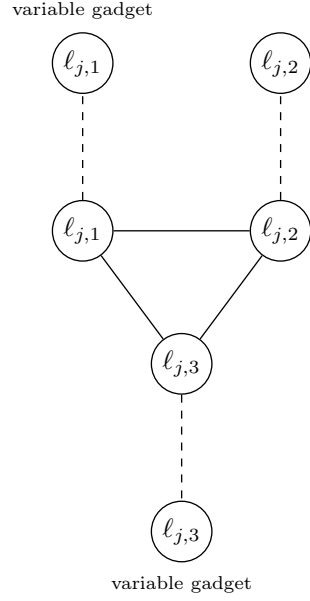
$$\phi = \bigwedge_{j=1}^c (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$$

be a 3CNF formula with variables $x_1, \dots, x_v$. We construct an undirected graph G as follows.

For each variable x_i , create two vertices labelled x_i and $\overline{x_i}$ and add the edge between them. This edge forces every vertex cover of the target size to choose at least one of the two literals.



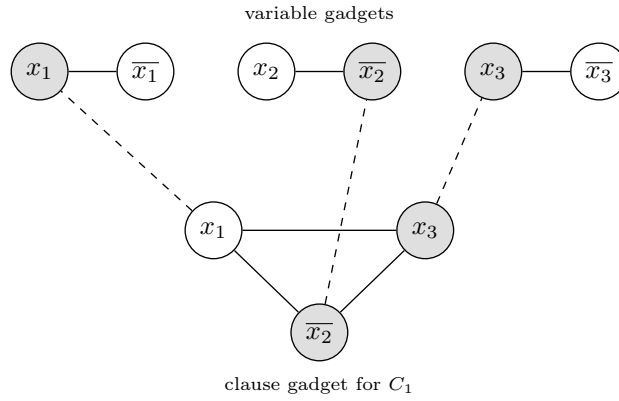
For each clause $C_j = (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$, create three vertices labelled $\ell_{j,1}, \ell_{j,2}, \ell_{j,3}$ and connect them as a triangle. Then connect each clause vertex $\ell_{j,r}$ to the vertex with the same literal label in the corresponding variable gadget.



For example, if a clause is

$$C_1 = (x_1 \vee \overline{x_2} \vee x_3),$$

then the corresponding part of the graph looks as follows. The dashed edges are the edges connecting the clause gadget to the variable gadgets.



In this example, the shaded vertices show the contribution of the satisfying assignment $x_1 = \text{true}$, $x_2 = \text{false}$, and $x_3 = \text{true}$: one vertex is chosen from each variable gadget, and two vertices are chosen from the clause triangle.

Finally, set

$$k = v + 2c.$$

The construction is polynomial, since it creates $2v + 3c$ vertices and $v + 6c$ edges.

If ϕ is satisfiable, take a satisfying assignment. For each variable gadget, put into the cover the vertex corresponding to the true literal: choose x_i if x_i is true, and choose $\overline{x_i}$ if x_i is false. This covers all variable-gadget edges. For each clause,

at least one literal is true. In the clause triangle, leave one true literal vertex unchosen and put the other two clause vertices into the cover. The two chosen vertices cover all three edges of the triangle. The edge from the unchosen true clause vertex to its variable gadget is covered by the already chosen true literal vertex in the variable gadget, and the other two connecting edges are covered by the two chosen clause vertices. Thus we obtain a vertex cover of size $v + 2c = k$.

Conversely, suppose G has a vertex cover C of size at most $k = v + 2c$. Each variable edge requires at least one chosen endpoint, so the v variable gadgets require at least v vertices. Each clause triangle requires at least two chosen vertices, so the c clause gadgets require at least $2c$ vertices. Since $|C| \leq v + 2c$, the cover must choose exactly one vertex from each variable gadget and exactly two vertices from each clause gadget.

Define a truth assignment by setting x_i to true iff the vertex x_i is chosen in the variable gadget. Consider any clause gadget. Since exactly two vertices of its triangle are chosen, one clause vertex is not chosen. The edge connecting this unchosen clause vertex to the corresponding variable gadget must still be covered, so the corresponding variable-gadget vertex is chosen. By the definition of the assignment, that literal is true. Hence every clause has at least one true literal, so ϕ is satisfiable.

Therefore

$$\phi \in 3SAT \iff \langle G, k \rangle \in \text{VERTEX COVER}.$$

Since $3SAT$ is NP-complete and the reduction is polynomial, VERTEX COVER is NP-hard. Together with VERTEX COVER being in NP, this proves that VERTEX COVER is NP-complete.